

SportsDeck Database Schema Documentation

Backend Design for a Sports Discussion Platform | Prisma ORM with SQLite

1. Introduction

SportsDeck is a social platform combining real-time sports data with community discussions. The schema supports three domains: **sports data** (teams, matches, standings), **social interactions** (threads, posts, polls), and **community moderation** (reports, bans, appeals). The complete ERD is provided in the attached `erd.png`

This document focuses on design decisions, trade-offs, and challenges encountered during development. The schema is implemented using Prisma ORM with SQLite, chosen for rapid development and deployment simplicity.

2. Domain Architecture Decisions

2.1 Sports Domain: External API Integration

Decision: Cache-First Architecture with Timestamps

All sports entities include a `cachedAt` field for time-based cache invalidation. This reduces API calls while managing data freshness via configurable TTL per entity type.

Challenge: Dual Identifier Management

Each entity maintains both internal `id` (for relationships) and `externalId` (for API sync). This dual-key approach complicates lookups but maintains referential integrity during external data reconciliation.

2.2 User Domain: Authentication Flexibility

Decision: Multi-Provider Authentication Schema

The User entity includes `authProvider` and `providerUserId` fields to support both local and OAuth authentication. The `passwordHash` is nullable for OAuth-only accounts.

Challenge: Ban State Consistency

The `isBanned` flag on User creates potential inconsistency with the separate Ban entity. We enforce consistency at the application layer through transaction wrappers.

2.3 Social Domain: Content Hierarchy

Decision: Three-Level Content Nesting

The schema implements Thread → Post → Reply hierarchy. This provides sufficient depth for sports discussions without arbitrary-depth nesting complexity. A recursive approach would offer more flexibility but complicates queries and pagination.

3. Key Design Challenges

3.1 Version History Implementation

Decision: Separate Version Tables

PostVersion and ReplyVersion tables store historical content. Each edit creates a new version record with previous content and timestamp.

Challenge: Storage Growth

Version history can grow rapidly in active communities. We considered limiting retention, compressing old versions, or archiving to separate storage. Currently all history is retained; future iterations may implement time-based archival.

Trade-off: Query Performance vs. Audit Completeness

Fetching posts with version history requires joins. We accept this overhead because auditability is critical for moderation. Storing versions as JSON would complicate querying specific versions.

3.2 Feed System Architecture

Decision: Pre-Aggregated Feed Events

The FeedEvent entity aggregates multiple Activity records into single feed items. For example, multiple replies by a user may be aggregated into one "User X was active in Thread Y" event.

Challenge: Fan-Out Problem

When a popular user posts, generating feed entries for all followers creates write amplification. Our current approach writes individual FeedEntry records per follower. For large user bases, this would require fan-out-on-read or hybrid approaches.

3.3 Poll Voting Integrity

Decision: Database-Level Vote Constraint

A composite unique constraint on (userId, pollOptionId) prevents duplicate votes at the database level, eliminating race condition vulnerabilities. This requires handling constraint violations in application code but is more reliable than application-level locking.

4. Moderation System Design

4.1 Report and Action Separation

Decision: Decoupled ReportedItem and Report

ReportedItem represents flagged content; Report represents individual user submissions. This allows multiple users to report the same item and aggregates counts for prioritization.

Challenge: Polymorphic Content References

ReportedItem must reference different content types. We use separate nullable foreign keys rather than a generic (type, id) pair, trading schema verbosity for referential integrity.

4.2 Ban and Appeal Workflow

Decision: Explicit Ban Records

Rather than simply toggling isBanned, we maintain full Ban records with reason, duration, and appeal status. This supports temporary bans, ban history review, and appeal workflows at the cost of record proliferation.

5. Normalization and Performance

5.1 Normalization Strategy

Decision: Third Normal Form (3NF) with Strategic Denormalization

The core schema follows 3NF to eliminate redundancy. We denormalize selectively: Standing entities duplicate team names to avoid joins in leaderboard queries, accepting update complexity for read performance.

5.2 Indexing Considerations

Decision: Composite Indexes for Common Queries

Key query patterns drove index design: Thread lookups by (teamId, createdAt), Post lookups by (threadId, createdAt) for pagination, and FeedEntry lookups by (userId, isRead) for notification counts.

Challenge: SQLite Index Limitations

SQLite's query planner is less sophisticated than PostgreSQL. Complex queries may require restructuring; we favor simpler, explicit queries over ORM-generated complex joins.

6. Lessons and Future Considerations

SQLite was chosen for development simplicity and single-file deployment. For production scale, migration to PostgreSQL would provide better concurrent write performance and JSONB support. The Prisma ORM abstraction makes such migration feasible without significant code changes.

Future enhancements include: (1) Full-text search for content discovery, (2) Notification preferences, (3) Media attachments, and (4) Real-time activity streaming. The current schema maintains extension points through nullable fields and junction tables.

Summary of Key Decisions

Table 1. Summary of Design Decisions

Area	Decision	Rationale
Sports Data	Cache with timestamps	API rate limit compliance
Authentication	Multi-provider in single table	Simpler queries, easier merging
Content	Three-level hierarchy	Sufficient depth, simpler queries
Versioning	Separate version tables	Audit completeness
Voting	DB-level unique constraint	Race condition prevention
Moderation	Decoupled report entities	Multi-user report aggregation
Feed	Pre-aggregated events	Read performance